



15. Neural Networks for Images

Using Deep Learning to solve complex Computer Vision problems

Previous Section:

 [14. Pre-Trained Neural Network Models](#)

Contents:

[1. Three Essential Computer Vision Tasks](#)

[2. Image Segmentation](#)

[2.1. Semantic Segmentation](#)

[2.2. Instance Segmentation](#)

[3. Image Segmentation with an Encoder-Decoder Model](#)

[3.1 Preparing Segmentation Data](#)

[3.2. Segmentation Encoder-Decoder Model](#)

[4. Image Segmentation with Modern CNN Architectures](#)

[4.1. Comparing Conv2D and SeparableConv2D](#)

[4.2. Residual Connections](#)

[4.3. Building a Small ResNet](#)

[4.4. Xception-Style Block](#)

[4.5. Building a Mini Xception Network](#)

[4.6. Visualizing Intermediate Activations](#)

[4.7. Visualizing Learned Filters](#)

[Summary](#)

[References](#)

In the previous chapters, we spent most of our time teaching machines how to recognize patterns, classify images, and make predictions. But eventually, we reach a point where traditional Machine Learning starts struggling. Some problems are simply too complex, too visual, and too dependent on spatial information for handcrafted features to handle effectively.

This is where Deep Learning truly begins to shine. When people hear the term *Computer Vision*, they often think about tasks such as recognizing faces, finding objects in photographs, or understanding scenes captured by cameras.

These problems require a machine not only to recognize what is present, but also to understand where it is and how different parts of an image relate to one another. Fortunately, Neural Networks—and especially Deep Neural Networks—are exceptionally good at learning these complex visual representations directly from data.

In this section, we will focus on the neural network architectures that made modern Computer Vision possible. We will not spend much time on practical image-processing techniques because many of those tasks are now efficiently handled by dedicated libraries such as OpenCV. That topic deserves an entire course of its own. For now, I want us to focus on the learning models themselves. How do neural networks move beyond simple image classification? How do they locate objects, separate foreground from background, or understand an entire scene?

To answer those questions, we first need to understand the major Computer Vision tasks that modern Deep Learning models are designed to solve.

1. Three Essential Computer Vision Tasks

Up to this point, most of our examples involved image classification. We give the model an image, and it tells us whether it contains a cat, a dog, a handwritten digit, or some other category. Simple enough.

But real-world images are rarely that cooperative. A photograph might contain multiple people, several vehicles, buildings, animals, and countless background objects all at the same time. In such situations, merely assigning a single label to the entire image is no longer sufficient. We need more detailed forms of understanding.

- The first and most fundamental task is **Image Classification**.

- Here, the model assigns one (***single-label multi-class classification***) or more labels (***multi-label classification***) to the entire image. The goal is to answer a simple question: ***“What is present in this image?”***
- The model does not care where the object appears. It only determines whether the object exists somewhere within the image.

However, there are many situations where knowing that an object exists is not enough. We may also want to know its location.

- This leads us to **Object Detection**.
 - Instead of only predicting labels, the model must identify individual objects and draw bounding boxes around them. In other words, it answers two questions simultaneously: ***What is the object?*** and ***Where is the object?***
 - A self-driving car, for example, cannot simply recognize that pedestrians exist somewhere in a camera frame. It must know their precise locations in order to react safely.

But even bounding boxes are sometimes too coarse. Suppose we want to distinguish every pixel belonging to a road, a tree, a building, or a person. A rectangular box cannot capture these fine details.

- This is where **Image Segmentation** becomes important.
 - Rather than assigning labels to entire images or rectangular regions, segmentation assigns labels to individual pixels. The model effectively creates a detailed map of the image, determining exactly which pixels belong to which objects.

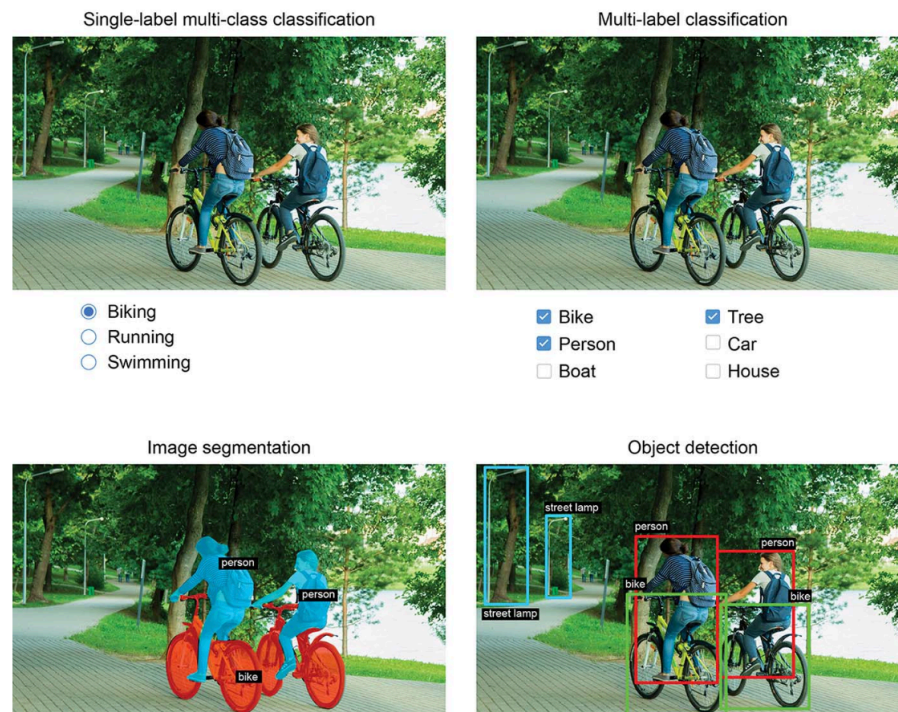


Figure 9.1 The three main computer vision tasks: classification, segmentation, detection

As we move from classification to detection and then to segmentation, something important happens. The output becomes increasingly detailed. And the more detailed the output becomes, the more important it is for the model to preserve spatial information throughout the network. Understanding *what* exists is no longer enough. The model must also understand *where* everything exists.

Of course, these are not the only Computer Vision problems that researchers work on. Many other specialized tasks exist:

- Image similarity and image retrieval
- Keypoint detection
- Human pose estimation
- Facial recognition
- Object tracking in videos
- 3D reconstruction and mesh prediction

Modern neural networks can even reconstruct three-dimensional structures from ordinary images or track moving objects across entire video sequences.

Despite their differences, all of these applications are built upon the same foundation. We still rely on four core ideas: Convolution; Representation Learning; Network Architecture; Interpretability

Everything we study from this point onward is ultimately an extension of these principles. The models may become larger, deeper, and more sophisticated, but the fundamental goal remains unchanged: ***Learning meaningful visual representations from data.***

2. Image Segmentation

Among the three major Computer Vision tasks, I personally find Image Segmentation the most fascinating. Classification tells us what is in an image. Object Detection tells us what is present and where it is located. Segmentation goes one step further by asking the model to understand every single pixel.

Instead of predicting a label for an entire image or drawing a rough bounding box around an object, a segmentation model must determine which category each pixel belongs to. In a sense, it transforms image understanding from object-level reasoning into pixel-level reasoning.

At first glance, Segmentation and Object Detection may seem very similar. After all, both attempt to identify and locate objects within an image. The difference lies in the level of detail.

An Object Detection model might draw a rectangle around a cat and declare, ***"There is a cat here"***. A Segmentation model, however, attempts to identify the exact shape of the cat by labeling every pixel that belongs to it. As a result, segmentation provides much more precise information about the scene.

There are two major forms of Image Segmentation that we should understand.

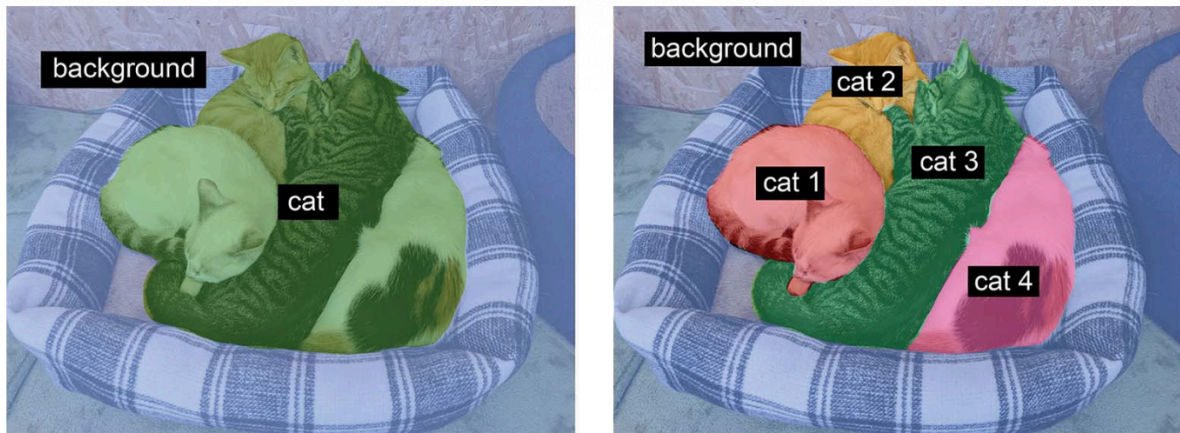


Figure 9.2 Semantic segmentation vs. instance segmentation

2.1. Semantic Segmentation

The first type is **Semantic Segmentation**. In Semantic Segmentation, each pixel is classified into a semantic category such as: Cat; Dog; Person; Road; Building; Tree.

The model does not distinguish between individual objects belonging to the same category. Its only goal is to determine what each pixel represents.

For example, imagine an image containing two cats. A semantic segmentation model would label all pixels belonging to both cats simply as **"cat"**. From the model's perspective: Cat A → Cat; Cat B → Cat. Both objects belong to the same semantic class, so they receive the same label.

This approach is extremely useful in applications such as autonomous driving, medical imaging, and satellite image analysis, where understanding regions and categories is often more important than identifying individual objects.

2.2. Instance Segmentation

The second type is **Instance Segmentation**. Instance Segmentation extends Semantic Segmentation by not only determining the category of each pixel, but also identifying which specific object instance those pixels belong to.

Returning to our example with two cats:

- Pixels belonging to the first cat are assigned to **Cat 1**

- Pixels belonging to the second cat are assigned to **Cat 2**

Even though both objects belong to the same category, the model treats them as separate entities.

This additional level of understanding becomes extremely valuable when we need to count objects, track them, or analyze individual characteristics. For example, in a wildlife monitoring system, Instance Segmentation can identify multiple cats separately, allowing us to classify not only that they are cats, but also distinguish their species, size, behavior, or movement individually.

As a result, Instance Segmentation combines the strengths of Object Detection and Semantic Segmentation. It knows: ***What the object is? Where the object is? Which pixels belong to the object? Which individual instance the object belongs to?*** Naturally, this makes Instance Segmentation significantly more challenging than Semantic Segmentation, but also far more powerful.

And as we move deeper into advanced neural network architectures, many of the most influential Computer Vision models were designed specifically to solve this pixel-level understanding problem.

3. Image Segmentation with an Encoder-Decoder Model

So far, our neural networks have mostly answered a simple question: ***“What is this image?”***. Image Segmentation asks a much harder question: ***“What is every pixel in this image?”***

That single difference changes everything. Instead of producing one prediction for an entire image, the network must generate thousands of predictions simultaneously—one for every pixel. A 96×96 image contains 9,216 pixels, meaning the model must classify 9,216 individual locations.

This is why segmentation models require a specialized architecture known as the **Encoder-Decoder Network**.

Import the required libraries:

```

import os
import random
import numpy as np
import matplotlib.pyplot as plt
from matplotlib import patches
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

SEED = 1337
random.seed(SEED)
np.random.seed(SEED)
tf.random.set_seed(SEED)

os.environ["TF_CPP_MIN_LOG_LEVEL"] = "2"

plt.rcParams["figure.figsize"] = (10, 4)
plt.rcParams["axes.grid"] = False

```

The line:

- `os.environ["TF_CPP_MIN_LOG_LEVEL"] = "2"` is used to control the verbosity of TensorFlow's logging.

- These lines:

```

plt.rcParams["figure.figsize"] = (10, 4)
plt.rcParams["axes.grid"] = False

```

set the default figure size for plots to 10 inches by 4 inches and disable the grid on the axes.

3.1 Preparing Segmentation Data

Before we can train a segmentation model, we need images and corresponding segmentation masks. Since collecting real-world segmentation datasets can be difficult, we will generate synthetic scenes automatically.

```

IMAGE_SIZE = 96
NUM_CLASSES = 3
CLASS_NAMES = ["background", "circle", "rectangle"]

# Each class is assigned a specific RGB color for visualization purposes.

```



```
# The colors are defined in the MASK_COLORS array, where ea
ch row corresponds to a class ID
# (0 for background, 1 for circle, and 2 for rectangle) and
contains the RGB values for that class.
```

```
MASK_COLORS = np.array(
    [
        [20, 20, 40],      # background: dark blue
        [255, 210, 70],    # circle: yellow orange
        [80, 200, 255],    # rectangle: light blue
    ],
    dtype="uint8",
)
```

```
def make_synthetic_scene(image_size=IMAGE_SIZE, seed=None):
    # Create a synthetic scene with random shapes and corre
sponding masks.
```

```
    rng = np.random.default_rng(seed)
```

```
    image = np.ones((image_size, image_size, 3), dtype="flo
at32") * 30
```

```
    image += rng.normal(loc=0, scale=4, size=image.shape)
```

```
    mask = np.zeros((image_size, image_size), dtype="uint
8")
```

```
    yy, xx = np.mgrid[:image_size, :image_size]
```

```
    boxes = []
```

```
    # Label 1 is assigned to circles, and label 2 is assign
ed to rectangles in the mask.
```

```
    radius = int(rng.integers(image_size // 10, image_size
// 6))
```

```
    cx = int(rng.integers(radius + 4, image_size - radius -
4))
```

```
    cy = int(rng.integers(radius + 4, image_size - radius -
4))
```

```

    circle_region = (xx - cx) ** 2 + (yy - cy) ** 2 <= radius
    us ** 2
    image[circle_region] = np.array([255, 210, 70])
    mask[circle_region] = 1
    boxes.append(("circle", cx - radius, cy - radius, cx +
radius, cy + radius))

    rect_w = int(rng.integers(image_size // 5, image_size
// 3))
    rect_h = int(rng.integers(image_size // 6, image_size
// 3))
    x1 = int(rng.integers(4, image_size - rect_w - 4))
    y1 = int(rng.integers(4, image_size - rect_h - 4))
    x2 = x1 + rect_w
    y2 = y1 + rect_h
    image[y1:y2, x1:x2] = np.array([80, 200, 255])
    mask[y1:y2, x1:x2] = 2
    boxes.append(("rectangle", x1, y1, x2, y2))

    # Convert the image to uint8 format and ensure that pixel
values are in the range [0, 255].
    image = np.clip(image, 0, 255).astype("uint8")
    return image, mask, boxes

def colorize_mask(mask):
    # Convert class IDs in the mask to RGB colors for visualization.
    # Use MASK_COLORS to map each class ID to an RGB color.
    return MASK_COLORS[mask]

```

```

# Sample image generation with make_synthetic_scene(seed=1
0).
image, mask, boxes = make_synthetic_scene(seed=10)

fig, axes = plt.subplots(1, 3, figsize=(15, 4))

# Classification: Only need class labels for the whole image

```

```

e.
axes[0].imshow(image)
axes[0].set_title("Classification\nscene contains circle +
rectangle")
axes[0].axis("off")

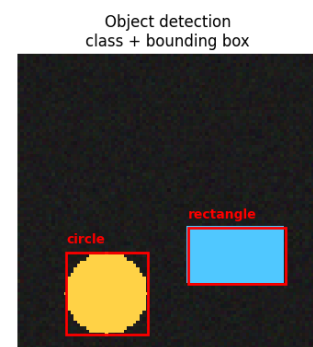
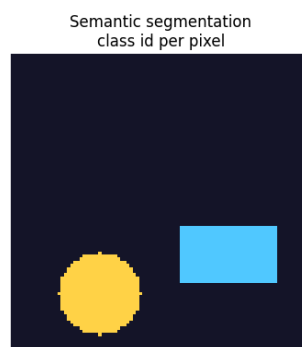
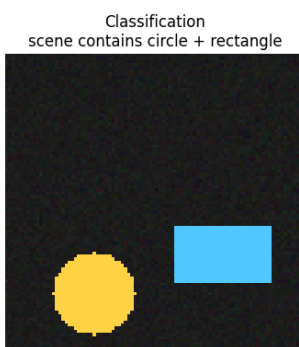
# Semantic segmentation: Require class id for each pixel
axes[1].imshow(colorize_mask(mask))
axes[1].set_title("Semantic segmentation\nclass id per pixel")
axes[1].axis("off")

# Object detection: Require bounding box
axes[2].imshow(image)
for class_name, x1, y1, x2, y2 in boxes:
    # Create Rectangle patch from bounding box.
    rect = patches.Rectangle((x1, y1), x2 - x1, y2 - y1,
                             linewidth=2, edgecolor="red",
                             facecolor="none")
    axes[2].add_patch(rect)
    axes[2].text(x1, y1 - 3, class_name, color="red", fontsize=10, weight="bold")

axes[2].set_title("Object detection\nclass + bounding box")
axes[2].axis("off")

plt.tight_layout()
plt.show()

```




```

def build_arrays(num_samples, seed_offset=0):
    # Generate images and masks to put into tf.data.Dataset
    t.
    images = []
    masks = []

    # Iterate through num_samples to generate image and mask
    for i in range(num_samples):
        image, mask, _ = make_synthetic_scene(seed=seed_offset + i)
        images.append(image)
        masks.append(mask[... , np.newaxis])

    images = np.array(images, dtype="uint8")
    masks = np.array(masks, dtype="uint8")
    return images, masks

TRAIN_SAMPLES = 240
VAL_SAMPLES = 60
BATCH_SIZE = 16

# Generate train_images/train_masks and val_images/val_masks.
train_images, train_masks = build_arrays(TRAIN_SAMPLES, seed_offset=0)
val_images, val_masks = build_arrays(VAL_SAMPLES, seed_offset=10_000)

# Generate train_ds from tensor_slices, shuffle, batch, prefetch.
train_ds = tf.data.Dataset.from_tensor_slices((train_images, train_masks))
train_ds = train_ds.shuffle(512, seed=SEED).batch(BATCH_SIZE).prefetch(tf.data.AUTOTUNE)

# Generate val_ds by from_tensor_slices, batch, prefetch.

```

```
val_ds = tf.data.Dataset.from_tensor_slices((val_images, val_masks))
val_ds = val_ds.batch(BATCH_SIZE).prefetch(tf.data.AUTOTUNE)

print("Train images:", train_images.shape)
print("Train masks:", train_masks.shape)
```

Train images: (240, 96, 96, 3)

Train masks: (240, 96, 96, 1)

Each image contains a circular object, a rectangular object, and a background region.

The segmentation mask assigns a class ID to every pixel:

Class ID	Class Name
0	Background
1	Circle
2	Rectangle

Unlike image classification datasets, where one image corresponds to one label, segmentation datasets require a mask with exactly the same spatial dimensions as the image.

For example: Image: $96 \times 96 \times 3 \rightarrow$ Mask: $96 \times 96 \times 1$. Each mask pixel stores the correct class for the corresponding image pixel.

- The function `make_synthetic_scene()` creates a synthetic RGB image, a semantic segmentation mask, and bounding boxes for visualization. This allows us to compare three Computer Vision tasks using the same scene:
 - Classification $\rightarrow$ image-level label
 - Object Detection $\rightarrow$ object-level bounding boxes
 - Segmentation $\rightarrow$ pixel-level labels
- The function `colorize_mask()` converts class IDs into colors so the segmentation results become easier to visualize. After generating many synthetic scenes, the images and masks are stored as NumPy arrays and converted into TensorFlow datasets.

- The datasets are then shuffled, batched, and prefetched to improve training efficiency.

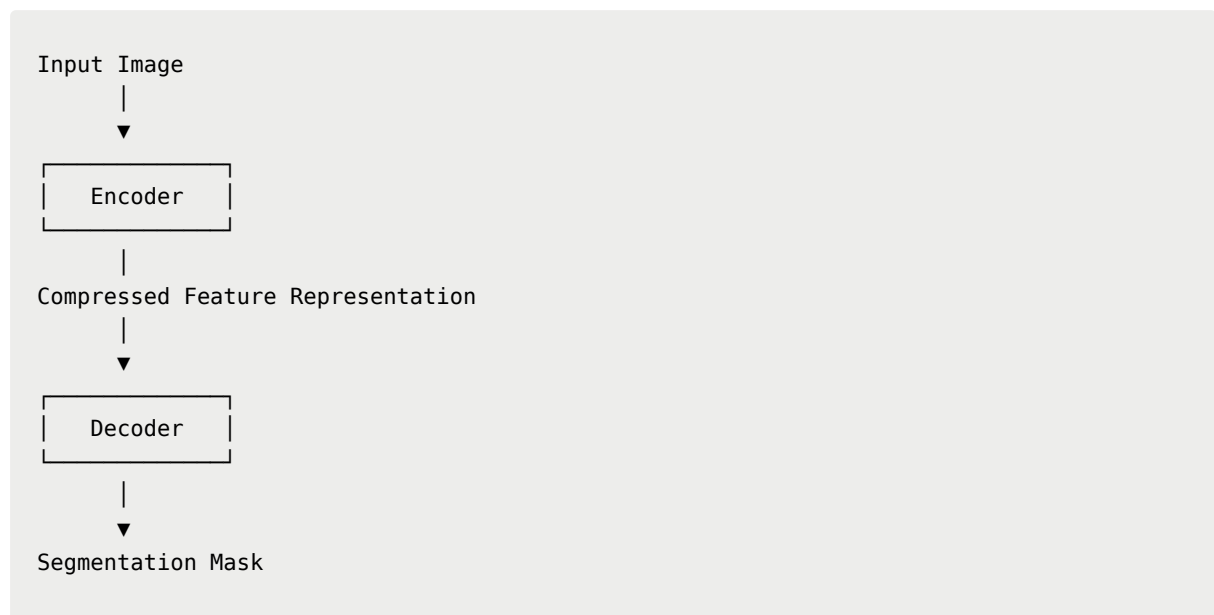
3.2. Segmentation Encoder-Decoder Model

Now we reach the most important part of this section. The challenge of segmentation is that the network must simultaneously understand what objects exist and preserve where those objects are located

Unfortunately, these goals often conflict with each other. To recognize objects effectively, neural networks prefer smaller feature maps because they contain more abstract information. To localize objects accurately, neural networks prefer larger feature maps because they preserve spatial details.

The Encoder-Decoder architecture solves this problem by splitting the network into two stages.

The Big Picture



The Encoder compresses information. The Decoder reconstructs spatial details. Together, they transform an image into a pixel-by-pixel prediction map.

```
def build_segmentation_model(image_size=IMAGE_SIZE, num_classes=NUM_CLASSES):  
    inputs = keras.Input(shape=(image_size, image_size, 3),
```



```

name="input_image")

# Rescale pixel from [0, 255] to [0, 1].
x = layers.Rescaling(1.0 / 255, name="rescaling")(inputs)

# Encoder rescale 96 -> 48 -> 24 -> 12.
x = layers.Conv2D(32, 3, strides=2, padding="same", activation="relu", name="encoder_conv_1")(x)
x = layers.Conv2D(64, 3, strides=2, padding="same", activation="relu", name="encoder_conv_2")(x)
x = layers.Conv2D(128, 3, strides=2, padding="same", activation="relu", name="encoder_conv_3")(x)

# Decode enlarge 12 -> 24 -> 48 -> 96.
x = layers.Conv2DTranspose(128, 3, strides=2, padding="same", activation="relu", name="decoder_deconv_1")(x)
x = layers.Conv2DTranspose(64, 3, strides=2, padding="same", activation="relu", name="decoder_deconv_2")(x)
x = layers.Conv2DTranspose(32, 3, strides=2, padding="same", activation="relu", name="decoder_deconv_3")(x)

# The last layer predicts num_classes for each pixel.
outputs = layers.Conv2D(num_classes, 3, padding="same", activation="softmax", name="pixel_classifier")(x)
return keras.Model(inputs, outputs, name="toy_segmentation_model")

seg_model = build_segmentation_model()

seg_model.compile(optimizer=keras.optimizers.Adam(learning_rate=1e-3),
                  loss="sparse_categorical_crossentropy",
                  metrics=["accuracy"],)

seg_model.summary()

```

- **Step 1 - Input Layer:** The model receives an RGB image with `inputs = keras.Input(shape=(image_size, image_size, 3), name="input_image")`
 - Each pixel contains three channels Red; Green; Blue.
 - The first operation rescales pixel values: `layers.Rescaling(1.0 / 255)` . Transforming: `0 → 255` into: `0.0 → 1.0`
 - This normalization helps training remain stable.
- **Step 2 - Encoder:** The Encoder gradually compresses the image. The key idea is `strides=2` . Every convolution cuts the spatial dimensions in half.
 - **Encoder Layer 1:** `Conv2D(32, 3, strides=2)` compresses `96 × 96 × 3 → 48 × 48 × 32` . The image becomes smaller, but the network learns 32 feature maps. These feature maps begin detecting edges, corners, and color transitions
 - **Encoder Layer 2:** `Conv2D(64, 3, strides=2)` compresses `48 × 48 × 32 → 24 × 24 × 64` . The spatial resolution decreases again. However, the feature count doubles. The network now learns more meaningful structures circular boundaries, rectangular edges, and object shapes
 - **Encoder Layer 3:** `Conv2D(128, 3, strides=2)` compresses `24 × 24 × 64 → 12 × 12 × 128` . At this stage, the original image has been heavily compressed. The network no longer stores individual pixels. Instead, it stores a compact representation describing the scene.

This compressed representation is sometimes called the **latent representation** or **bottleneck**. You can think of it as a summary of everything important in the image.

Consider how humans recognize objects. When you look at a cat, you do not memorize every pixel. Instead, your brain extracts higher-level concepts fur, ears, tail, and body shape

The Encoder performs a similar process. It gradually removes unnecessary detail while preserving useful semantic information. The result is a compact feature representation that is much easier for the network to reason about.

- **Step 3: Decoder:** Now comes the difficult part. The Encoder compressed the image from 96×96 to 12×12 . But segmentation requires predictions for every pixel. We therefore need to reconstruct the original resolution. This is the Decoder's job.
 - The Decoder uses: `Conv2DTranspose`, which is often called a **transposed convolution** or **deconvolution**. Unlike regular convolutions that shrink feature maps, transposed convolutions expand them.
 - **Decoder Layer 1:** `Conv2DTranspose(128, 3, strides=2)` reconstructs $12 \times 12 \times 128 \rightarrow 24 \times 24 \times 128$
 - **Decoder Layer 2:** `Conv2DTranspose(64, 3, strides=2)` reconstructs $24 \times 24 \times 128 \rightarrow 48 \times 48 \times 64$
 - **Decoder Layer 3:** `Conv2DTranspose(32, 3, strides=2)` reconstructs $48 \times 48 \times 64 \rightarrow 96 \times 96 \times 32$

The Decoder gradually restores spatial resolution. You can think of it as rebuilding the image from the compressed representation generated by the Encoder.

However, unlike image generation, the Decoder is reconstructing a semantic understanding of the scene rather than the original pixels.

- **Step 4: Pixel Classifier**

After reconstruction, every pixel still needs a class prediction. The final layer is:

`Conv2D(3, 3, activation="softmax")` → Output: $96 \times 96 \times 3$

Notice that the final channel count equals the number of classes background, circle, or rectangle. For every pixel, the model outputs three probabilities:

Example: `Pixel (40, 20) → Background: 0.01; Circle: 0.97; Rectangle: 0.02`

The largest probability becomes the predicted class using `np.argmax`, the network converts probabilities into class IDs.

Why Encoder-Decoder Models Are Important

This simple architecture demonstrates the fundamental idea behind modern segmentation networks. Many famous segmentation architectures are built on the same principle: FCN (Fully Convolutional Network); U-Net; SegNet; DeepLab; Mask R-CNN segmentation branch

Although these models become much larger and more sophisticated, they all follow the same philosophy:

1. Compress the image into meaningful features.
2. Learn a rich latent representation.
3. Reconstruct spatial information.
4. Predict a class for every pixel.

In other words, image classification asks: ***"What is in this image?"***, object detection asks: ***"What objects are present and where are they?"***, segmentation asks: ***"What is every single pixel?"***. And the Encoder-Decoder architecture is one of the most elegant solutions ever developed for answering that question.

Finally, we simply train the model and observe the result:

```
EPOCHS = 4

# EarlyStopping is used to stop training if the validation
loss stops improving
callbacks = [keras.callbacks.EarlyStopping(monitor="val_loss",
patience=2,
                                                    restore_best_weights=True,)]

# Train the model with train_ds and val_ds.
history = seg_model.fit(train_ds, validation_data=val_ds,
                        epochs=EPOCHS, callbacks=callbacks,)
```

Plot training history:

```
def plot_training_history(history):
    metrics = history.history
    epochs = range(1, len(metrics["loss"]) + 1)
```

```

fig, axes = plt.subplots(1, 2, figsize=(12, 4))

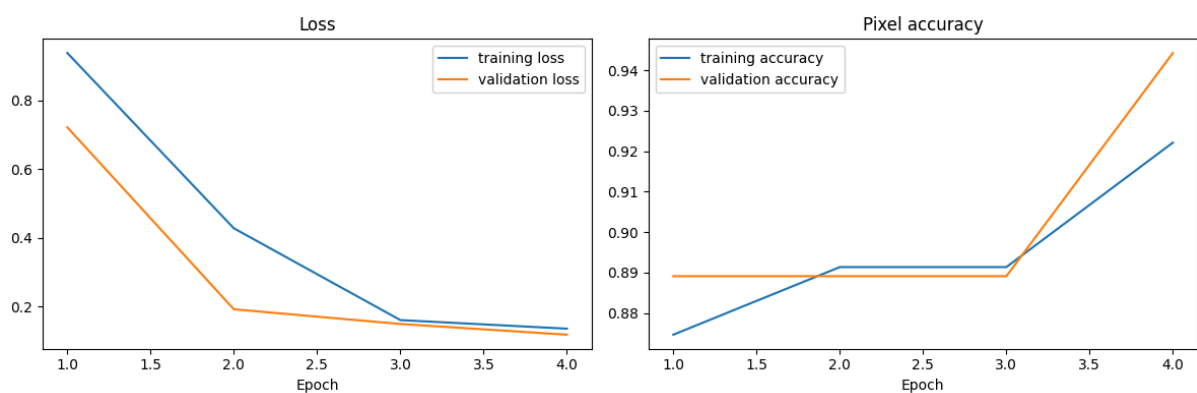
axes[0].plot(epochs, metrics["loss"], label="training loss")
axes[0].plot(epochs, metrics["val_loss"], label="validation loss")
axes[0].set_title("Loss")
axes[0].set_xlabel("Epoch")
axes[0].legend()

axes[1].plot(epochs, metrics["accuracy"], label="training accuracy")
axes[1].plot(epochs, metrics["val_accuracy"], label="validation accuracy")
axes[1].set_title("Pixel accuracy")
axes[1].set_xlabel("Epoch")
axes[1].legend()

plt.tight_layout()
plt.show()

plot_training_history(history)

```



Sample mask prediction:

```

# Use a batch validation to see how the model predict mask
sample_images, sample_masks = next(iter(val_ds))

# Prediction for the first three samples
pred_probs = seg_model.predict(sample_images[:3])

# Use argmax in class to get predicted
pred_masks = np.argmax(pred_probs, axis=-1).astype("uint8")

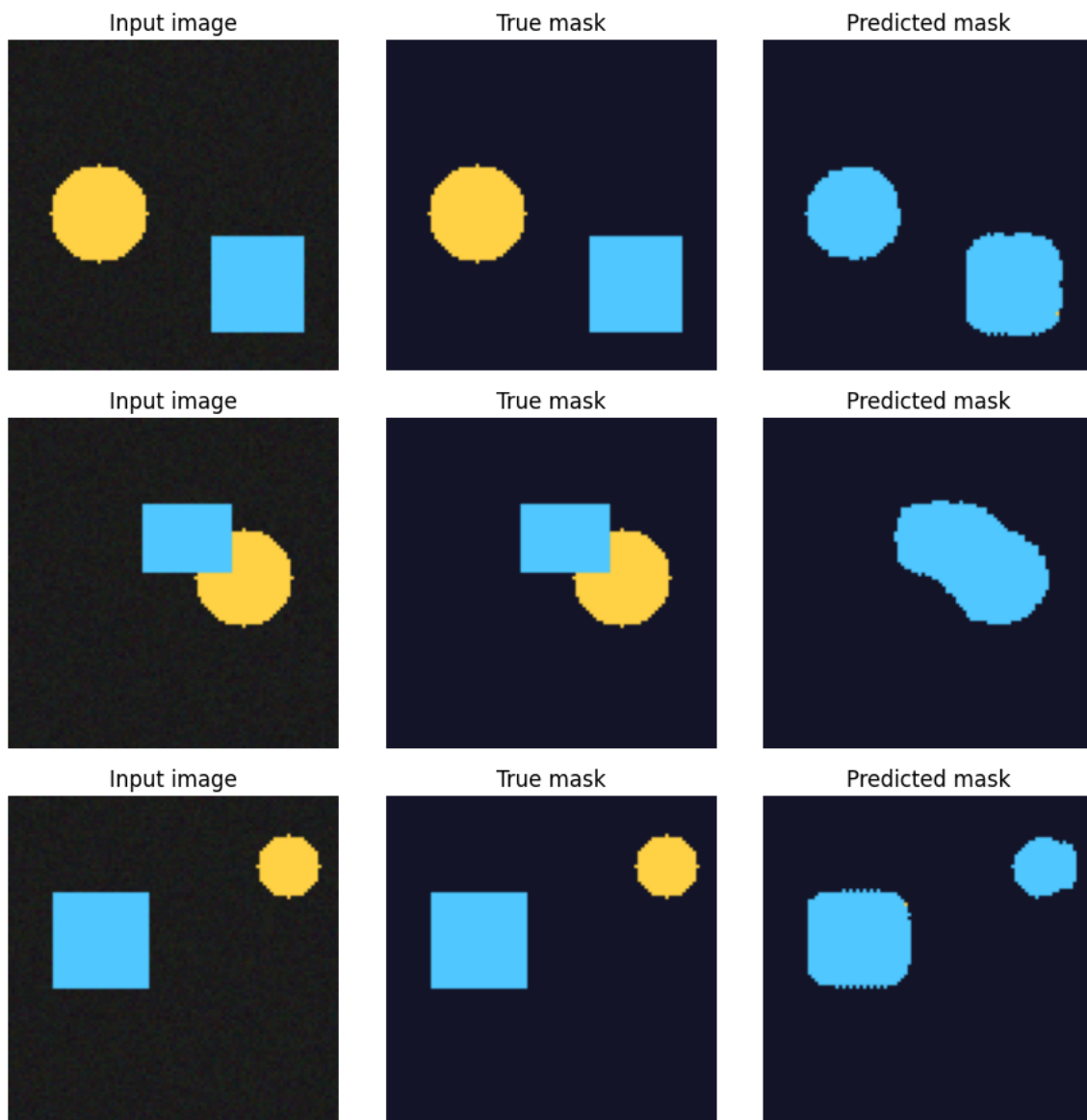
fig, axes = plt.subplots(3, 3, figsize=(9, 9))
for i in range(3):
    axes[i, 0].imshow(sample_images[i].numpy().astype("uint8"))
    axes[i, 0].set_title("Input image")
    axes[i, 0].axis("off")

    true_mask = sample_masks[i].numpy().squeeze()
    axes[i, 1].imshow(colorize_mask(true_mask))
    axes[i, 1].set_title("True mask")
    axes[i, 1].axis("off")

    axes[i, 2].imshow(colorize_mask(pred_masks[i]))
    axes[i, 2].set_title("Predicted mask")
    axes[i, 2].axis("off")

plt.tight_layout()
plt.show()

```



4. Image Segmentation with Modern CNN Architectures

The Encoder-Decoder model from the previous section successfully demonstrated how segmentation works. However, modern convolutional neural networks rarely rely on plain convolution layers alone.

As neural networks became deeper, researchers encountered three major challenges: too many parameters, difficult optimization in deep networks, inefficient feature extraction

To address these issues, modern architectures introduced three important ideas: **Separable Convolutions; Batch Normalization; and Residual Connections**

Many influential architectures such as Xception, ResNet, DeepLab, and MobileNet are built upon these concepts.

4.1. Comparing Conv2D and SeparableConv2D

```
def count_params_for_layer(layer):
    inputs = keras.Input(shape=(32, 32, 64))
    model = keras.Model(inputs, layer(inputs))
    return model.count_params()

standard_params = count_params_for_layer(layers.Conv2D(64,
3, padding="same",
                                                    use_
bias=False))

separable_params = count_params_for_layer(layers.SeparableC
onv2D(64, 3, padding="same",

use_bias=False))

print("Standard Conv2D parameters:", standard_params)
print("SeparableConv2D parameters:", separable_params)
print("Parameter reduction:", round(standard_params / separ
able_params, 2), "times")
```

Standard Conv2D parameters: 36864

SeparableConv2D parameters: 4672

Parameter reduction: 7.89 times

Here we compare the number of trainable parameters required by: Conv2D and SeparableConv2D.

Assume: `Input channels = 64; Output channels = 64; Kernel size = 3×3` . A standard convolution requires: `3 × 3 × 64 × 64 = 36,864 parameters`

A SeparableConv2D performs two operations:

- **Step 1 - Depthwise Convolution:** Apply one filter per input channel. For instance, $3 \times 3 \times 64 = 576$ parameters
- **Step 2 - Pointwise Convolution:** Combine channels using 1×1 convolutions. For instance, $1 \times 1 \times 64 \times 64 = 4,096$ parameters
- As a result, Total: $576 + 4,096 = 4,672$ parameters . This dramatically reduces memory and computation.

4.2. Residual Connections

One of the most important inventions in deep learning is the residual connection.

```

def residual_block(x, filters, stride=1, name="residual_block"):
    residual = x

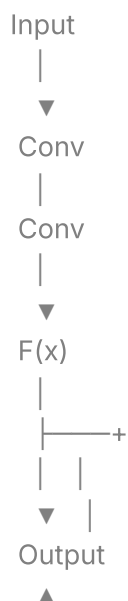
    x = layers.Conv2D(filters, 3, strides=stride, padding="same",
                      use_bias=False, name=f"{name}_conv1")(x)
    x = layers.BatchNormalization(name=f"{name}_bn1")(x)
    x = layers.Activation("relu", name=f"{name}_relu1")(x)

    x = layers.Conv2D(filters, 3, padding="same",
                      use_bias=False, name=f"{name}_conv2")(x)
    x = layers.BatchNormalization(name=f"{name}_bn2")(x)

    if residual.shape[-1] != filters or stride != 1:
        residual = layers.Conv2D(filters, 1, strides=stride, padding="same",
                                  use_bias=False, name=f"{name}_projection")(residual)

    x = layers.add([x, residual], name=f"{name}_add")
    return layers.Activation("relu", name=f"{name}_out")(x)

```



|
x

The network learns: $F(x)$ instead of $H(x) \rightarrow$ The final output becomes: $y = F(x) + x$, where: x = shortcut connection; $F(x)$ = learned transformation.

Deep networks often suffer from vanishing gradients, exploding gradients, or degradation problem. The shortcut connection creates a direct path for gradients.

This allows information to flow through very deep networks.

- Without residual connections:
 - 20 layers $\rightarrow$ difficult
 - 50 layers $\rightarrow$ very difficult
 - 100+ layers $\rightarrow$ unstable
- With residual connections:
 - 50 layers
 - 101 layers
 - 152 layers
 - Each of which becomes practical.

4.3. Building a Small ResNet

```

inputs = keras.Input(shape=(64, 64, 3), name="input")

x = layers.Rescaling(1.0 / 255, name="rescaling")(inputs)
x = layers.Conv2D(32, 3, padding="same",
                  use_bias=False, name="stem_conv")(x)
x = layers.BatchNormalization(name="stem_bn")(x)
x = layers.Activation("relu", name="stem_relu")(x)

x = residual_block(x, 32, stride=1, name="block1")
x = residual_block(x, 64, stride=2, name="block2")

# Classification head.
x = layers.GlobalAveragePooling2D(name="global_average_pooling")(x)
outputs = layers.Dense(3, activation="softmax", name="classifier")(x)

residual_demo_model = keras.Model(inputs, outputs,
                                   name="residual_demo_model")

residual_demo_model.summary()

```

Architecture

```

Input
↓
Conv
↓
Residual Block
↓
Residual Block
↓
Global Average Pooling
↓
Dense
↓
Softmax

```

This demonstrates the core idea behind ResNet.

4.4. Xception-Style Block

Xception replaces regular convolutions with `SeparableConv2D` .

```

# Mini Xception block using depthwise separable convolution
# S.
def xception_like_block(x, filters, stride=2, name="xception_like_block"):
    residual = x

    x = layers.BatchNormalization(name=f"{name}_bn1")(x)
    x = layers.Activation("relu", name=f"{name}_relu1")(x)
    x = layers.SeparableConv2D(filters, 3, padding="same",
                               use_bias=False, name=f"{name}_sepconv1")(x)

    x = layers.BatchNormalization(name=f"{name}_bn2")(x)
    x = layers.Activation("relu", name=f"{name}_relu2")(x)
    x = layers.SeparableConv2D(filters, 3, padding="same",
                               use_bias=False, name=f"{name}_sepconv2")(x)

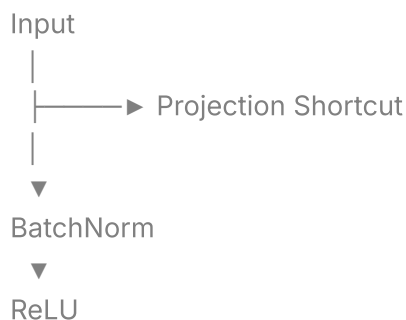
    x = layers.MaxPooling2D(3, strides=stride,
                           padding="same", name=f"{name}_pool")(x)

    residual = layers.Conv2D(filters, 1, strides=stride,
                             padding="same", use_bias=False,
                             name=f"{name}_projection")(residual)

    return layers.add([x, residual], name=f"{name}_add")

```

Architecture



▼
SeparableConv
▼
BatchNorm
▼
ReLU
▼
SeparableConv
▼
MaxPool
|
▼
Add

This is essentially a simplified version of the Xception architecture.

4.5. Building a Mini Xception Networ

```

# Mini Xception-style classifier using separable convolution
n blocks.
inputs = keras.Input(shape=(96, 96, 3), name="input")

# Input preprocessing and entry layer.
x = layers.Rescaling(1.0 / 255, name="rescaling")(inputs)
x = layers.Conv2D(32, 5, padding="same",
                  use_bias=False, name="entry_conv")(x)

# Xception feature extraction blocks.
for filters in [32, 64, 128]:
    x = xception_like_block(x, filters,
                            name=f"xception_block_{filter
s}")

# Classification head.
x = layers.GlobalAveragePooling2D(name="global_average_pool
ing")(x)
x = layers.Dropout(0.3, name="dropout")(x)
outputs = layers.Dense(2, activation="softmax",
                       name="classifier")(x)

xception_demo_model = keras.Model(inputs,
                                   outputs, name="mini_xcept
ion_like_model")

xception_demo_model.summary()

```

Architecture

```

Input
↓
Conv
↓
Xception Block
↓
Xception Block
↓
Xception Block
↓

```


Global Average Pooling

↓

Dropout

↓

Softmax

This network combines batch Normalization, separable convolution, and residual connections, which are three major innovations used in modern CNNs.

4.6. Visualizing Intermediate Activations

One advantage of CNNs is that we can inspect what the network sees internally.

```
# Intermediate Activations
test_image = sample_images[:1]

layer_names = ["encoder_conv_1", "encoder_conv_2",
               "encoder_conv_3", "decoder_deconv_1"]

layer_outputs = [seg_model.get_layer(name).output
                  for name in layer_names]

activation_model = keras.Model(seg_model.input, layer_outputs)
activations = activation_model.predict(test_image)

for layer_name, activation in zip(layer_names, activations):
    num_channels = min(8, activation.shape[-1])
    fig, axes = plt.subplots(1, num_channels, figsize=(2 *
num_channels, 2))
    fig.suptitle(f"Intermediate activations - {layer_name}", y=1.05)

    for channel_index in range(num_channels):
        # Take a 2D feature map 2D and normalize
        channel_image = activation[0, :, :, channel_index]
```

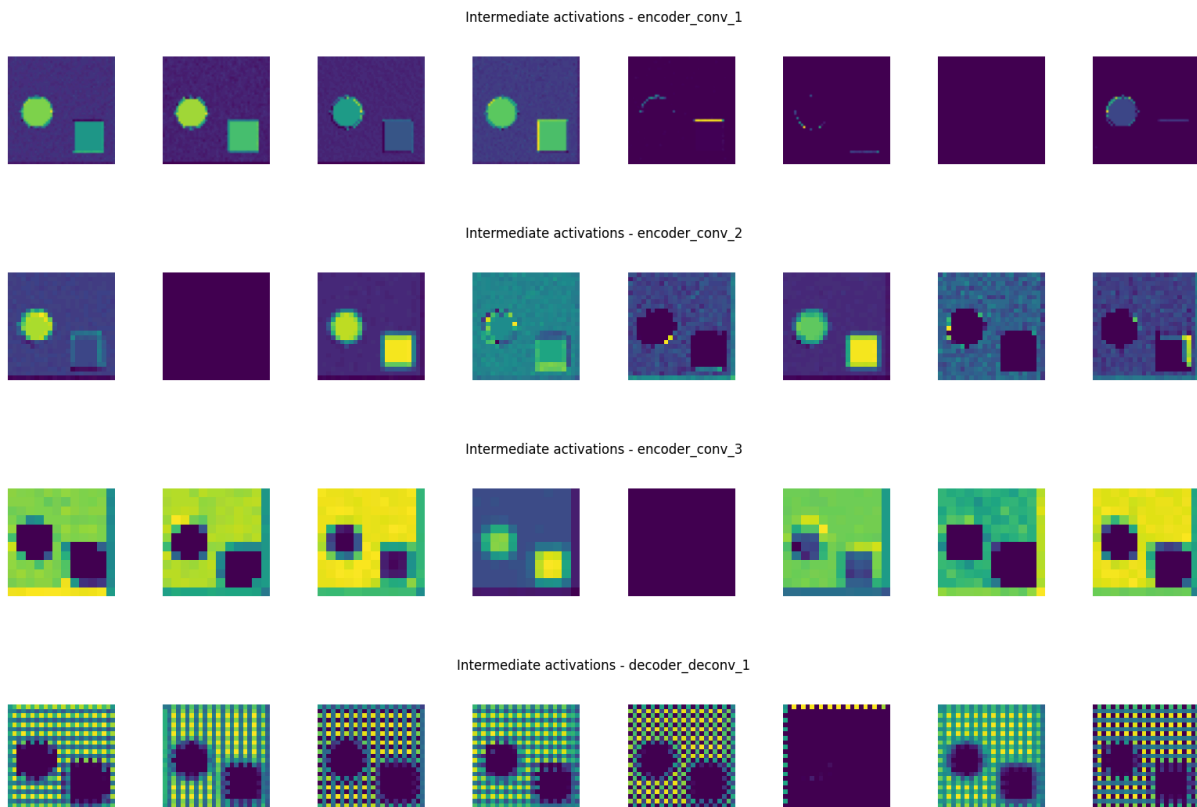
```

channel_image = channel_image - channel_image.mean
()
channel_image = channel_image / (channel_image.std
() + 1e-6)

axes[channel_index].imshow(channel_image, cmap="vir
idis")
axes[channel_index].axis("off")

plt.tight_layout()
plt.show()

```



The model returns feature maps from multiple layers.

- **Encoder Layer 1:** $96 \times 96 \times 32$ → Feature maps detect edges, boundaries, and simple textures. The outputs still resemble the original image.
- **Encoder Layer 2:** $48 \times 48 \times 64$ → Feature maps begin detecting circles, rectangles, and larger structures. More semantic information appears.

- **Encoder Layer 3:** $24 \times 24 \times 128$ → The network becomes highly abstract. Many channels no longer resemble the original image. Instead, they represent object presence, shape concepts, and region importance
- **Decoder Layer 1:** $24 \times 24 \times 128$ → The network begins reconstructing spatial information. Shapes become more recognizable again.

4.7. Visualizing Learned Filters

The final experiment visualizes what a filter has learned.

```
def deprocess_image(x):
    x = x - x.mean()
    x = x / (x.std() + 1e-6)
    x = x * 64 + 128
    x = np.clip(x, 0, 255).astype("uint8")
    return x

def generate_filter_pattern(model, layer_name, filter_index
=0, steps=40, learning_rate=10.0):
    # Input image as filter, strongly activated with gradient ascent
    # Create feature_extractor to return activation of target layer
    feature_extractor = keras.Model(model.input, model.get_layer(layer_name).output)

    # Random image within [0, 255].
    image = tf.Variable(tf.random.uniform([1, IMAGE_SIZE, IMAGE_SIZE, 3], minval=0, maxval=255))

    for step in range(steps):
        with tf.GradientTape() as tape:
            activation = feature_extractor(image)
            loss = tf.reduce_mean(activation[:, :, :, filter_index])
```

```

        # Compute and normalize gradient, update image the
        o gradient ascent.
        grads = tape.gradient(loss, image)
        grads = grads / (tf.sqrt(tf.reduce_mean(tf.square(g
rads)))) + 1e-6)
        image.assign_add(learning_rate * grads)
        image.assign(tf.clip_by_value(image, 0, 255))

    return deprocess_image(image.numpy()[0])

pattern = generate_filter_pattern(seg_model, layer_name="en
coder_conv_1", filter_index=0)

plt.figure(figsize=(4, 4))
plt.imshow(pattern)
plt.title("Filter visualization: encoder_conv_1 / filter
0")
plt.axis("off")
plt.show()

```

Filter visualization: encoder_conv_1 / filter 0



The algorithm uses Gradient Ascent. Instead of changing model weights (**Weights fixed** → **Input image changes**). The image is modified repeatedly to maximize the activation of a specific filter.

Filters in early layers often respond to horizontal edges, vertical edges, diagonal lines, color transitions. Filters in deeper layers often respond to object parts, shapes, complex patterns

This provides a rare glimpse into what the neural network has actually learned. For that reason, activation visualization and filter visualization are often considered the first steps toward understanding the interpretability of deep neural networks.

Summary

In this section, we explored how advanced neural network architectures are applied to Computer Vision problems, with a particular focus on **Image Segmentation**. Unlike image classification, which predicts a single label for an entire image, segmentation requires the model to classify every pixel individually, making it one of the most challenging Computer Vision tasks.

- We began by generating a synthetic dataset containing circles and rectangles along with their corresponding semantic segmentation masks. This allowed us to clearly illustrate the differences between image classification, object detection, and image segmentation while keeping the dataset simple enough to understand.
- Next, we introduced the **Encoder-Decoder architecture**, one of the fundamental designs used in segmentation networks. The Encoder progressively compressed the input image into a compact feature representation using convolutional layers, while the Decoder reconstructed spatial information through transposed convolutions to produce a pixel-wise prediction map. This architecture demonstrated how neural networks can simultaneously learn semantic information and preserve object locations.
- After understanding the basic segmentation pipeline, we examined several innovations that power modern convolutional neural networks.
 - We studied **Separable Convolutions**, which dramatically reduce the number of trainable parameters by separating spatial filtering from channel mixing.
 - We then explored **Batch Normalization**, which stabilizes activations during training and allows deeper networks to converge more reliably.

- Finally, we investigated **Residual Connections**, one of the most influential ideas in deep learning, which enables information and gradients to flow directly through shortcut paths and makes very deep networks practical to train.

Using these building blocks, we constructed simplified versions of modern architectures inspired by **ResNet** and **Xception**, demonstrating how contemporary CNNs combine efficiency, stability, and representational power. We also visualized intermediate activations and learned filters to better understand what the network learns at different depths. Early layers were shown to focus on low-level features such as edges and textures, while deeper layers captured increasingly abstract concepts related to object shapes and semantic regions.

Overall, this section illustrates the evolution from basic convolutional networks to modern Computer Vision architectures. The Encoder-Decoder model provides the foundation for segmentation, while techniques such as Separable Convolutions, Batch Normalization, and Residual Connections form the core components behind many state-of-the-art models used in image segmentation, object detection, and other advanced visual understanding tasks.

References

<https://www.geeksforgeeks.org/computer-vision/image-segmentation-techniques-and-applications/>

<https://www.geeksforgeeks.org/deep-learning/convolutional-neural-network-cnn-in-machine-learning/>

<https://www.ultralytics.com/blog/perceptrons-and-neural-networks-basic-principles-of-computer-vision>

Next Section:

 [16. Recurrent Neural Networks](#)